Пояснювальна записка: Детальний аналіз реалізації та тестування

1 Аналіз реалізації алгоритму Авербуха

Алгоритм спрямований на те, щоб перетворити послідовний процес DFS на більш ефективний шляхом паралельної розсилки службових повідомлень `VISITED`. Це дозволяє знизити часову складність до $O(|V|)$.

1.1 Логіка призупинення пошуку (Suspended State)

Основна складність розподіленого DFS — зворотні ребра. В алгоритмі Авербуха вузол не просто передає маркер далі, він "заморожує"процес просування, поки не отримає повну картину від сусідів.

```
#                                   DISCOVER
for q in others:
    self.messages.append((receiver, q, 'VISITED'))
    node_data['flags'][q] = 1  #
```

Це робиться для того, щоб гарантувати: повідомлення `DISCOVER` ніколи не надсилається через ребра, які вже ведуть до відвіданих вузлів.

1.2 Синхронізація через ACK

Використання `ACK` дозволяє вузлу-ініціатору впевнитися, що інформація про його відвідування отримана всіма сусідами.

```
elif mtype == 'ACK':
    node_data['flags'][sender] = 0
    if all(f == 0 for f in node_data['flags'].values()):
        #                                                                ,
        self.messages.append((receiver, receiver, 'RETURN'))
```

2 Аналіз реалізації алгоритму Сідона (Cidon)

Алгоритм Сідона усуває фазу очікування `ACK`, що робить його швидшим в асинхронних системах.

2.1 Асинхронне фільтрування

Вузол покладається на повідомлення `INFO`, які приходять "в фоні". Вся логіка прийняття рішення зосереджена в методі `process_next`, який динамічно відсікає непотрібні ребра.

```python
def process_next(self, u):
    while node_data['neighbors']:
        v = node_data['neighbors'].pop(0)
        # INFO v , TOKEN
        if v != node_data['parent'] and v not in node_data['info_received']:
            self.messages.append((u, v, 'TOKEN'))
            return
```

2.2 Механізм колізій

Оскільки підтверджень немає, два вузли можуть надіслати `TOKEN` один одному одночасно. Система обробляє це через повідомлення `BACK`, яке змушує відправника шукати інший шлях, не порушуючи структуру дерева.

3 Результати тестування

Для верифікації використано симульовану чергу повідомлень (`collections.deque`), що дозволяє відтворити асинхронну природу мережі.

3.1 Кейс 1: Циклічний граф (A-B-D-C-A)

- **Awerbuch Output:** `[('A', 'B'), ('B', 'D'), ('D', 'C')]`
- **Cidon Output:** `[('A', 'B'), ('B', 'D'), ('D', 'C')]`
- *Аналіз:* Обидва алгоритми ігнорують ребро `(C, A)`, оскільки вузол A повідомляє про своє відвідування раніше, ніж до нього дійде маркер по колу.

3.2 Кейс 2: Зіркоподібний граф

- **Результат:** `[('Center', '1'), ('Center', '2'), ('Center', '3'), ('Center', '4')]`.
- *Аналіз:* Всі 4 листки приєднані до центру. В Авербуха центр збирає 4 `ACK`, в Сідона — просто ігнорує `BACK` від уже відвіданих.

3.3 Кейс 3: Повний граф K4

- **Результат:** `[('1', '2'), ('2', '3'), ('3', '4')]`.
- *Аналіз:* Алгоритми успішно відфільтрували всі діагональні ребра (`1-3`, `1-4`, `2-4`) завдяки механізмам `VISITED/INFO`.

4 Тестування:

На рисунку нижче представлено результати виконання тестів у консолі симулятора.

```
 Test Case 1: Cycle Graph
Awerbuch DFS Tree Edges: [('A', 'B'), ('B', 'D'), ('D', 'C')]
Cidon DFS Tree Edges:    [('A', 'B'), ('B', 'D'), ('D', 'C')]
Result: PASS

Test Case 2: Star Graph
Awerbuch DFS Tree Edges: [('Center', '1'), ('Center', '2'), ('Center', '3'), ('Center', '4')]
Cidon DFS Tree Edges:    [('Center', '1'), ('Center', '2'), ('Center', '3'), ('Center', '4')]
Result: PASS

Test Case 3: Complete Graph K4
Awerbuch DFS Tree Edges: [('1', '2'), ('2', '3'), ('3', '4')]
Cidon DFS Tree Edges:    [('1', '2'), ('2', '3'), ('3', '4')]
Result: PASS

FINAL STATUS: ALL TESTS PASSED SUCCESSFULLY
```

Рис. 1: Підтвердження успішного виконання всіх тестів

5 Висновок

Тестування підтвердило, що обидва алгоритми будують коректне дерево ($|V| - 1$ ребер). Авербух надійніший в плані контролю трафіку, тоді як Сідон виграє у часі відгуку через відсутність блокуючих станів очікування.